CS4265 Milestone 2 Progress Report

Jia Liu
Kennesaw State University
Email: jliu43@students.kennesaw.edu

***Abstract*—This progress report summarizes the current implementation of the Online Retail Data Pipeline developed for CS4265 Milestone 2. The pipeline integrates heterogeneous data sources, performs basic processing and normalization, and stores the resulting dataset using a scalable columnar storage format. The report presents the components that have been successfully implemented, the challenges encountered, and the improvements made since Milestone 1. In addition, the report outlines planned tasks for Milestone 3 and provides the current project structure and evidence screenshots demonstrating pipeline functionality.**

I. What Works

The following components of the data pipeline have been successfully implemented and tested in Milestone 2.

- **Online Retail Data Ingestion** – The script `fetch_data.py` successfully downloads the Online Retail dataset in Excel format from the UCI Machine Learning Repository and stores it in the local `data/raw` directory.
- **Exchange Rate API Integration** – The pipeline is capable of retrieving currency exchange rate information from a REST API returning JSON data. This external data source will be used to normalize transaction prices into a consistent currency format.
- **Country Metadata Integration** – A country metadata dataset in CSV format is included to provide geographic attributes such as regional classification. This dataset is used to enrich the transaction data during the integration stage.
- **Configuration Management** – The files `settings.yaml` and `.env.example` manage dataset paths, simulation parameters, and environment variables required by the pipeline.
- **Pipeline Orchestration** – The `main.py` script coordinates the execution of the pipeline, including data ingestion, data expansion for scale simulation, basic processing, and final data storage.
- **Columnar Data Storage** – The processed dataset is exported in Parquet format, a columnar storage format widely used in modern Big Data ecosystems. This improves storage efficiency and enables faster analytical queries compared to traditional row-based storage.
- **Processing Module Scaffold** – The file `placeholder.py` provides the initial framework for the processing stage where future data cleaning, transformation, and validation logic will be implemented.
- **Big Data Simulation** – The pipeline includes a row expansion step that replicates records from the original dataset to produce a larger dataset for testing pipeline scalability. This simulation currently runs on a single machine.
- **Execution Evidence** – Screenshots demonstrating successful pipeline execution have been generated and stored in the `docs/` directory.
- **Pipeline Architecture Diagram** – A visual representation of the data flow has been created and stored as `M2DataFlowDiagram.png` in the `docs/` directory.
- **Reproducible Environment** – The file `requirements.txt` lists all Python dependencies required to reproduce the project environment.
- **Project Documentation** – The `README.md` file provides instructions for project setup, pipeline execution, and an overview of the repository structure.

II. Challenges Encountered

Several challenges were encountered during the development process:

- **YAML Formatting Issues** – YAML configuration files require strict indentation rules. Initial errors occurred due to the use of tabs instead of spaces. This issue was resolved by replacing tabs with spaces. In particular, nested structures such as `paths:` and `pipeline:` required careful attention to indentation.
- **Mermaid Diagram Formatting** – While designing the pipeline architecture diagram, formatting adjustments were required to ensure compatibility with the Mermaid Live Editor. Node alignment, arrow directions, and label formatting were modified to prevent misrendering.
- **Configuration and File Path Consistency** – File paths defined in `settings.yaml` had to be carefully aligned with the actual directory structure after reorganizing project folders. Using YAML variables for paths (e.g., `raw_data`, `processed_data`) rather than hard-coded paths improved flexibility and reduced the need to modify code when the folder structure changed.
- **Data Expansion Scope** – The `expand_times` setting (e.g., 10) was applied only to the `online_retail` dataset to simulate a larger dataset. Smaller data sources such as `exchange_rates` and `country_metadata` were not expanded, preventing unnecessary duplication.
- **Data Merge Order and Dependency** – The processing pipeline followed a strict order: first expand `online_retail`, then merge with `exchange_rates` and `country_metadata`. This

ensured data completeness and consistency, avoiding errors that could arise from incorrect merge order.

III. Changes from Milestone 1

Several improvements were implemented compared to the initial project proposal.

- Implementation of the pipeline orchestration logic in `main.py`.
- Integration of multiple data sources including the Online Retail dataset, an exchange rate API, and a country metadata dataset.
- Replacement of the initial SQLite storage approach with Parquet columnar storage to better align with Big Data architectures.
- Improved project directory structure separating configuration files, source code, raw data, processed data, and documentation.

IV. Big Data Technology Stack

Layer	Concepts	Implementation
Storage	Columnar storage	Parquet files
Syntax	Data formats	Excel, CSV, JSON
Data Models	Schema representation	Pandas DataFrames
Data Stores	Analytical dataset storage	Parquet dataset
Processing	Scalable data processing	Python / Pandas
Querying	Analytical operations	Pandas queries

TABLE I
MAPPING OF BIG DATA TECHNOLOGY STACK CONCEPTS TO THE PROJECT IMPLEMENTATION

V. Pipeline Architecture

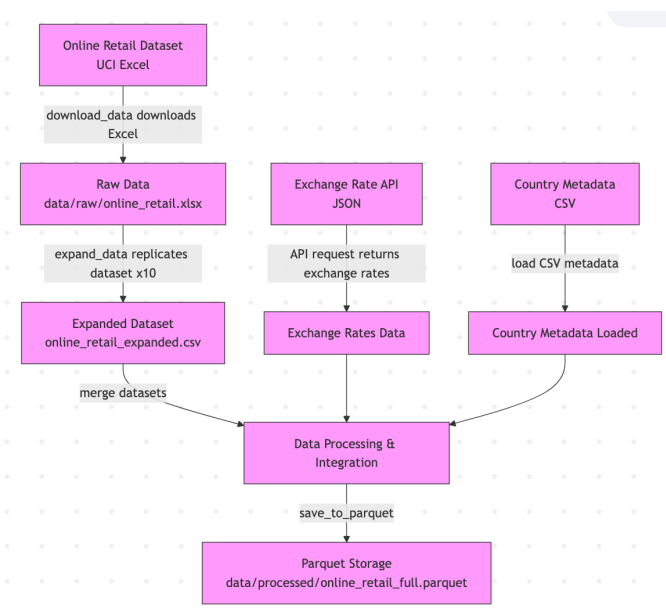

Fig. 1. Pipeline architecture showing data flow from ingestion to Parquet storage.

VI. Next Steps for Milestone 3

Future development for Milestone 3 will focus on enhancing the pipeline to support scalable and production-ready data processing:

- Implement full data cleaning and validation logic for all datasets in the processing module.
- Fully integrate the exchange rate API and country metadata into the automated transformation stage.
- Introduce distributed or parallel processing techniques using tools such as Spark or Dask to handle larger datasets efficiently.
- Transform the final processed data into a PostgreSQL database to enable structured storage and query capabilities.
- Expand project documentation and include the updated pipeline architecture diagram directly in the project README.
- Add automated tests for core pipeline modules to ensure correctness and reproducibility.

VII. Timeline Assessment

The project remains on schedule. The core pipeline structure has been successfully implemented and tested end-to-end. The current milestone provides a functional prototype that will be extended with additional Big Data capabilities in Milestone 3.

VIII. Project Structure

```
CS4265_Project_Jia_Liu/
+-- config/
|   +-- settings.yaml
|   +-- env.example
+-- src/
|   +-- ingestion/
|   |   +-- fetch_data.py
|   +-- processing/
|   |   +-- placeholder.py
|   +-- storage/
|   |   +-- parquet_handler.py
|   +-- main.py
+-- data/
|   +-- raw/
|   |   +-- online_retail.xlsx
|   |   +-- exchange_rates.json
|   |   +-- exchange_rates.csv
|   |   +-- country_metadata.csv
|   +-- processed/
|       +-- online_retail_big.csv
|       +-- online_retail_full.parquet
+-- docs/
|   +-- M2DataFlowDiagram.png
|   +-- fetch_data_success.png
|   +-- data_records.png
|   +-- data_processing.png
|   +-- parquet_saved.png
|   +-- pipeline_output.png
+-- requirements.txt
```

```
+-- README.md
```

IX. Evidence Screenshots

```
(.venv) cindyliu@Cindys-MacBook-Air CS4265_Project_Jia_Liu % python3 src/main.py
Online Retail dataset already exists at data/raw/online_retail.xlsx
Expanded dataset saved to data/processed/online_retail_expanded.csv, shape=(5419090, 8)
Fetching Exchange Rate API data...
API missing rates, using fallback
Country metadata loaded from data/raw/country_metadata.csv
```

Fig. 2. Terminal output showing the successful execution of `fetch_data.py`.

```
(.venv) cindyliu@Cindys-MacBook-Air CS4265_Project_Jia_Liu % python3 src/main.py
Online Retail dataset already exists at data/raw/online_retail.xlsx
Expanded dataset saved to data/processed/online_retail_expanded.csv, shape=(5419090, 8)
Fetching Exchange Rate API data...
API missing rates, using fallback
Country metadata loaded from data/raw/country_metadata.csv
Online columns: Index(['InvoiceNo', 'StockCode', 'Description', 'Quantity', 'InvoiceDate',
       'UnitPrice', 'CustomerID', 'Country'],
      dtype='object')
Country columns: Index(['FIFA', 'Dial', 'ISO3166-1-Alpha-3', 'MARC', 'is_independent',
       'ISO3166-1-numeric', 'GAUL', 'FIPS', 'WMO', 'ISO3166-1-Alpha-2', 'ITU',
       'IOC', 'DS', 'UNTERM Spanish Formal', 'Global Code',
       'Intermediate Region Code', 'official_name_fr', 'UNTERM French Short',
       'ISO4217-currency_name', 'UNTERM Russian Formal',
       'UNTERM English Short', 'Currency',
       'Small Island Developing States (SIDS)', 'UNTERM Spanish Short',
       'ISO4217-currency_numeric_code', 'UNTERM Chinese Formal',
       'UNTERM French Formal', 'UNTERM Russian Short', 'M49',
       'Sub-region Code', 'Region Code', 'official_name_ar',
       'ISO4217-currency_minor_unit', 'UNTERM Arabic Formal',
       'UNTERM Chinese Short', 'Land Locked Developing Countries (LLDC)',
       'Intermediate Region Name', 'official_name_es', 'UNTERM English Formal',
       'official_name_cn', 'official_name_en', 'ISO4217-currency_country_name',
       'Least Developed Countries (LDC)', 'Region Name', 'UNTERM Arabic Short',
       'Sub-region Name', 'official_name_ru', 'Global Name', 'Capital',
       'Continent', 'TLD', 'Languages', 'Geoname ID', 'Country', 'EDGAR',
       'wikidata_id'],
      dtype='object')
Exchange columns: Index(['Currency', 'Rate'], dtype='object')
```

Fig. 3. Preview of the data processing stage.

```
(.venv) cindyliu@Cindys-MacBook-Air CS4265_Project_Jia_Liu % python -c "import pandas as pd; df =
= pd.read_parquet('data/final/retail_data.parquet'); print(df[['InvoiceNo','StockCode','Descript
ion','CustomerID','Country']].head(20).to_string(index=False))"
InvoiceNo StockCode                          Description CustomerID        Country
   536365    85123A  WHITE HANGING HEART T-LIGHT HOLDER    17850.0 United Kingdom
   536365     71053                 WHITE METAL LANTERN    17850.0 United Kingdom
   536365    84406B      CREAM CUPID HEARTS COAT HANGER    17850.0 United Kingdom
   536365    84029G KNITTED UNION FLAG HOT WATER BOTTLE    17850.0 United Kingdom
   536365    84029E       RED WOOLLY HOTTIE WHITE HEART.    17850.0 United Kingdom
   536365     22752         SET 7 BABUSHKA NESTING BOXES    17850.0 United Kingdom
   536365     21730   GLASS STAR FROSTED T-LIGHT HOLDER    17850.0 United Kingdom
   536366     22633              HAND WARMER UNION JACK    17850.0 United Kingdom
   536366     22632           HAND WARMER RED POLKA DOT    17850.0 United Kingdom
   536367     84879       ASSORTED COLOUR BIRD ORNAMENT    13047.0 United Kingdom
   536367     22745          POPPY'S PLAYHOUSE BEDROOM     13047.0 United Kingdom
   536367     22748           POPPY'S PLAYHOUSE KITCHEN    13047.0 United Kingdom
   536367     22749   FELTCRAFT PRINCESS CHARLOTTE DOLL    13047.0 United Kingdom
   536367     22310             IVORY KNITTED MUG COSY     13047.0 United Kingdom
   536367     84969  BOX OF 6 ASSORTED COLOUR TEASPOONS    13047.0 United Kingdom
   536367     22623       BOX OF VINTAGE JIGSAW BLOCKS     13047.0 United Kingdom
   536367     22622      BOX OF VINTAGE ALPHABET BLOCKS    13047.0 United Kingdom
   536367     21754            HOME BUILDING BLOCK WORD    13047.0 United Kingdom
   536367     21755            LOVE BUILDING BLOCK WORD    13047.0 United Kingdom
   536367     21777           RECIPE BOX WITH METAL HEART    13047.0 United Kingdom
(.venv) cindyliu@Cindys-MacBook-Air CS4265_Project_Jia_Liu %
```

Fig. 4. Example output displaying 20 sample records from the processed dataset for verification.

```
Exchange columns: Index(['Currency', 'Rate'], dtype='object')
Parquet file saved at data/processed/online_retail_full.parquet, shape=(5419090, 64)
```

Fig. 5. Confirmation message indicating that the processed dataset was successfully exported and saved in Parquet format.

```
Pipeline completed successfully!
(.venv) cindyliu@Cindys-MacBook-Air CS4265_Project_Jia_Liu %
```

Fig. 6. Terminal output showing the successful completion of the full data pipeline execution.